CACS355 · BCA VI Semester · Network Programming

Unit 3: URLs and URIs

Theory, exam programs and interview notes, in **Java** (for the exam) and **Go** (for jobs), side by side.

Syllabus topic	Theory	Java	Go
3.1 URIs: URLs and relative URLs	A1, A2	-	-
3.2 The URL class: creating, retrieving data, splitting, equality and conversion	A1	B1-B5	C1-C5
3.3 The URI class: constructing, parts, resolving, equality and strings	A1, A2	B6-B9	C6-C9
3.4 x-www-form-urlencoded: URLEncoder and URLDecoder	A3	B10	C10
3.5 Proxies: system properties, Proxy and ProxySelector	A4	B11, B12	C11, C12
3.6 Communicating with server-side programs through GET	A5	B13	C13
3.7 Password-protected sites: Authenticator, PasswordAuthentication, JPasswordField	A6	B14, B15	C14, C15

How to read this guide

Part A, Theory is shared by both languages. **Part B, Java** is the exam track. **Part C, Go** is the job track.

Part D lists likely exam questions; **Part E** is the Java vs Go cheat sheet.

Read B and C in parallel. The section numbers match: B2 goes with C2, B3 with C3, and so on.

Programs are in the `java` and `go` folders next to this PDF. All were compiled and run, and the outputs shown are real.

Sections 3.5 to 3.7 need a server, so a small one is included: `LocalTestServer` (Java) and `testserver` (Go). Both run on port 8000 and can be swapped, so no outside website is needed.

Unit 3 has 5 teaching hours, one of the heaviest units. The class and method tables, URL splitting, URLEncoder and Authenticator are the most likely sources of questions.

Contents

Part A: Theory (common to Java and Go) **4**

A1. URI, URL and URN (3.1) . . . 4

A2. Relative URLs (3.1) . . . 5

A3. URL encoding and x-www-form-urlencoded (3.4) . . . 5

A4. Proxy servers (3.5) . . . 5

A5. Communicating with server-side programs through GET (3.6) . . . 6

A6. Password-protected sites (3.7) . . . 6

Part B: Java (exam track) **8**

B1. The URL class . . . 8

B2. Creating new URLs (3.2) . . . 8

B3. Retrieving data from a URL (3.2) . . . 10

B4. Splitting a URL into pieces (3.2) . . . 11

B5. Equality, comparison and conversion (3.2) . . . 12

B6. The URI class: constructing a URI (3.3) . . . 13

B7. The parts of the URI (3.3) . . . 14

B8. Resolving relative URIs (3.3) . . . 15

B9. Equality, comparison and string representation (3.3) . . . 16

B10. URLEncoder and URLDecoder (3.4) . . . 17

B11. Proxies: system properties and the Proxy class (3.5) . . . 18

B12. The ProxySelector class (3.5) . . . 20

B13. Communicating with server-side programs through GET (3.6) . . . 21

B14. Authenticator and PasswordAuthentication (3.7) . . . 22

B15. The JPasswordField class (3.7) . . . 24

B16. Tips for Unit 3 in the exam . . . 25

Part C: Go (job track) **26**

C1. The net/url package . . . 26

C2. Creating URLs (Java B2) . . . 26

C3. Retrieving data (Java B3) . . . 28

C4. Splitting a URL (Java B4) . . . 29

C5. Equality and conversion (Java B5) . . . 30

C6. Constructing a URI (Java B6) . . . 30

C7. The parts of a URI (Java B7) . . . 31

C8. Resolving relative URIs (Java B8) . . . 32

C9. Equality and string form (Java B9) . . . 33

C10. Encoding and query strings (Java B10) . . . 34

C11. Proxies (Java B11) . . . 35

C12. Choosing a proxy per request (Java B12) . . . 36
C13. GET requests with a query (Java B13) . . . 37
C14. Password-protected sites (Java B14) . . . 39
C15. Reading a password (Java B15) . . . 40
C16. Interview notes (Go) . . . 40
Part D: Likely exam questions for Unit 3 **41**
Short questions . . . 41
Programs (long questions) . . . 41
Part E: Java vs Go cheat sheet (Unit 3) **43**

Part A: Theory (common to Java and Go)

A1. URI, URL and URN (3.1)

A **URI (Uniform Resource Identifier)** is a string that identifies a resource. It has the general form `scheme:scheme-specific-part`. There are two kinds:

- **URL (Uniform Resource Locator):** a URI that says **where** a resource is and how to get it, e.g. `https://www.example.com/index.html`.
- **URN (Uniform Resource Name):** a URI that gives a resource a permanent **name** without saying where it is, e.g. `urn:isbn:9781449357672`.

Every URL is a URI, but not every URI is a URL.

Basis	URI	URL	URN
Meaning	Identifies a resource	Locates a resource	Names a resource
Relationship	The general term	A type of URI	A type of URI
Tells you how to fetch it?	Not necessarily	Yes	No
Example	tel:+977-1-4331000	http://example.com/a.html	urn:isbn:9781449357672
Java class	java.net.URI	java.net.URL	java.net.URI
Go type	url.URL	url.URL	url.URL (Opaque field)

Parts of a URL

```
https://admin@www.example.com:8443/docs/guide.html?lang=en&v=2#intro
\___/   \___/ \_____________/ \__/\_____________/ \_________/ \___/
scheme  user       host      port      path           query  fragment
        \_______authority________/
```

Part	In the example	Meaning
Scheme / protocol	https	How to reach the resource: http, https, ftp, file, mailto...
User info	admin	Optional login name (a password here is unsafe and discouraged)
Host	www.example.com	Name or IP address of the server
Port	8443	Optional; defaults come from the scheme (http 80, https 443, ftp 21)
Authority	admin@www.example.com:8443	User info + host + port together
Path	/docs/guide.html	Where the resource is on that server
Query	lang=en&v=2	Extra data for a server-side program, after ?
Fragment	intro	A place inside the document, after #. **Never sent to the server.**

Hierarchical and opaque URIs

- **Hierarchical:** the part after the scheme starts with / and has a tree structure, e.g. `http://example.com/a/b`.
- **Opaque:** the part after the scheme does not start with / and cannot be split further, e.g. `mailto:someone@example.com` or `tel:+977-1-4331000`.

A2. Relative URLs (3.1)

An **absolute URL** contains everything needed to find the resource. A **relative URL** leaves parts out and inherits them from a **base URL**, normally the page that contains the link. Relative links keep working when a whole site is moved to another host.

Base URL used below: `http://www.example.com/docs/intro/index.html`

Relative URL	Rule	Result
chapter2.html	Replaces the last path segment	http://www.example.com/docs/intro/chapter2.html
../images/logo.png	.. goes up one directory	http://www.example.com/docs/images/logo.png
/about.html	Leading / starts at the server root	http://www.example.com/about.html
//cdn.example.org/a.js	Leading // keeps only the scheme	http://cdn.example.org/a.js
#top	Same document, new fragment	http://www.example.com/docs/intro/index.html#top
https://other.org/x	Already absolute: base is ignored	https://other.org/x

A3. URL encoding and x-www-form-urlencoded (3.4)

A URL can only contain a limited set of ASCII characters. Anything else must be **percent-encoded**: each byte becomes % followed by two hex digits. In UTF-8, é is two bytes, so it becomes %C3%A9.

Group	Characters	Treatment
Unreserved	A-Z a-z 0-9 - _ . ~	Never need encoding
Reserved	: / ? # [ ] @ ! $ & ' ( ) * + , ; =	Have special meaning, and must be encoded when used as ordinary data
Everything else	space, % < > " non-ASCII letters...	Must always be encoded

application/x-www-form-urlencoded is the format browsers use to send form data: `name=value&name=value`. Names and values are encoded separately, and a **space becomes +**.

> **Most common mistake**
>
> Encode each **name and value** separately, never the whole URL. Encoding the whole URL also turns :// and ? and & into %XX, which destroys its structure (see the Wrong/Right lines in B10).

A4. Proxy servers (3.5)

A **proxy server** sits between a client and the Internet. The client sends its request to the proxy, and the proxy forwards it and passes back the response.



Why use a proxy	How
Access control	An organisation allows or blocks sites (common in colleges and offices)

Why use a proxy	How
Caching	Popular pages are stored, saving bandwidth and time
Security and logging	Only the proxy touches the Internet, and traffic can be scanned and logged
Privacy	The web server sees the proxy's IP address, not the client's

Proxy type	Works at	Java constant	Notes
HTTP	Application layer	Proxy.Type.HTTP	Understands HTTP; can cache and filter
SOCKS	Session layer	Proxy.Type.SOCKS	Forwards any TCP traffic without understanding it
Direct	No proxy	Proxy.Type.DIRECT	Connects to the server directly

With an HTTP proxy, the first request line contains the **full URL** (`GET http://www.example.com/page.html HTTP/1.1`) instead of only the path. That is how the test servers in this unit detect that they are being used as a proxy.

A5. Communicating with server-side programs through GET (3.6)

A web form sends its data to a server-side program. With the **GET** method, the data travels in the query string of the URL, so a program can talk to the server just by opening a URL. No form or browser is needed.

Steps

- Find the program's address (the form's **action** attribute).
- Find the input names (each `<input name=...>` in the form) and check that **method** is GET.
- Encode each name and value with URLEncoder, and join them with &.
- Append ? and the query string to the address, open the URL and read the response.

Basis	GET	POST (Unit 4/5)
Where the data goes	In the URL query string	In the request body
Visible in browser/logs	Yes	No
Size limit	Limited by URL length	Practically none
Use for	Safe reads: search, filters	Changes: login, orders, uploads
Can be bookmarked/cached	Yes	No

A6. Password-protected sites (3.7)

Websites protect pages in two main ways. **Cookie-based form login** uses an HTML form and is what most sites use (Unit 4). **HTTP authentication** is built into the protocol, and it is what Java's Authenticator handles.

HTTP Basic authentication flow

#	Direction	Message
1	Client to server	`GET /secret HTTP/1.1`
2	Server to client	`401 Unauthorized` with `WWW-Authenticate: Basic realm="Network Programming"`
3	Client	Gets a user name and password (Java calls the Authenticator here)
4	Client to server	`GET /secret` again, with `Authorization: Basic c3R1ZGVudDpuZXR3b3JrMTIz`

#	Direction	Message
5	Server to client	`200 OK` and the page

> **Base64 is not encryption**
>
> The value after **Basic** is only base64 of `user:password`, which anyone can decode. Basic authentication is only safe over HTTPS. **Digest** authentication avoids sending the password itself, but HTTPS with Basic or tokens is the usual modern choice.

Part B: Java (exam track)

B1. The URL class

- `java.net.URL` is a **final** class (it cannot be extended) that represents a URL and can fetch what it points to.
- **Immutable:** there are no setters, so URL objects are thread-safe.
- It uses the **strategy design pattern**: a **protocol handler** does the real work for each protocol (http, ftp, file...).
- An invalid URL or a protocol with no handler throws the checked `MalformedURLException`.
- Only the syntax and protocol are checked at construction. No network connection is made.

> **Newer Java versions**
>
> The URL constructors shown here are deprecated from Java 20, in favour of `new URI(...).toURL()`. They still work, and the syllabus and textbook use them, so write them in the exam.

B2. Creating new URLs (3.2)

Constructor	Use
`URL(String url)`	One complete absolute URL
`URL(String protocol, String host, String file)`	From parts, default port
`URL(String protocol, String host, int port, String file)`	From parts, explicit port
`URL(URL base, String relative)`	Relative URL resolved against a base

`URLConstructors.java`

```
import java.net.*;

// 3.2 Creating new URLs: the URL constructors.
public class URLConstructors {
    public static void main(String[] args) {
        try {
            // 1. from one complete string
            URL u1 = new URL("http://www.example.com:8080/docs/index.html?lang=en#top");
            System.out.println("URL(spec)                    : " + u1);

            // 2. from protocol, host and file
            URL u2 = new URL("http", "www.example.com", "/docs/index.html");
            System.out.println("URL(protocol, host, file)    : " + u2);

            // 3. from protocol, host, port and file
            URL u3 = new URL("https", "www.example.com", 8443, "/docs/index.html");
            System.out.println("URL(protocol,host,port,file) : " + u3);

            // 4. relative to a base URL
            URL base = new URL("http://www.example.com/docs/intro/index.html");
            System.out.println("URL(base, chapter1.html)     : " + new URL(base, "chapter1.html"));
            System.out.println("URL(base, ../img/logo.png)   : " + new URL(base, "../img/logo.png"));
            System.out.println("URL(base, /about.html)       : " + new URL(base, "/about.html"));

            // an unknown protocol throws MalformedURLException
            URL bad = new URL("htp://www.example.com/");
            System.out.println("Never printed: " + bad);
        } catch (MalformedURLException e) {
            System.out.println("Malformed URL: " + e.getMessage());
        }
```

```
    }
}
```

Sample output

```
URL(spec)                    : http://www.example.com:8080/docs/index.html?lang=en#top
URL(protocol, host, file)    : http://www.example.com/docs/index.html
URL(protocol,host,port,file) : https://www.example.com:8443/docs/index.html
URL(base, chapter1.html)     : http://www.example.com/docs/intro/chapter1.html
URL(base, ../img/logo.png)   : http://www.example.com/docs/img/logo.png
URL(base, /about.html)       : http://www.example.com/about.html
Malformed URL: unknown protocol: htp
```

Which protocols does the JVM support?

Since new `URL()` throws MalformedURLException when no handler exists, trying each protocol is a simple test.

ProtocolTester.java

```
import java.net.*;

// 3.2 Which protocols does this virtual machine support?
public class ProtocolTester {
    public static void main(String[] args) {
        String[] addresses = {
            "http://www.example.com/", "https://www.example.com/", "ftp://ftp.example.com/",
            "file:///C:/temp/a.txt", "jar:http://www.example.com/a.jar!/b.class",
            "mailto:someone@example.com", "telnet://example.com/", "gopher://example.com/",
            "ldap://ldap.example.com/", "rmi://example.com/", "jdbc:mysql://localhost/db"
        };
        for (String address : addresses) {
            test(address);
        }
    }

    private static void test(String address) {
        try {
            URL u = new URL(address);
            System.out.println(u.getProtocol() + " is supported");
        } catch (MalformedURLException e) {
            System.out.println(address.substring(0, address.indexOf(':')) + " is NOT supported");
        }
    }
}
```

Sample output (JDK 11)

```
http is supported
https is supported
ftp is supported
file is supported
jar is supported
mailto is supported
telnet is NOT supported
gopher is NOT supported
ldap is NOT supported
rmi is NOT supported
jdbc is NOT supported
```

B3. Retrieving data from a URL (3.2)

Method	Returns	Use
`openStream()`	InputStream	Raw body of the resource; the simplest and most common
`openConnection()`	URLConnection	Lets you set request headers and read response headers (Unit 5)
`openConnection(Proxy p)`	URLConnection	Same, through a chosen proxy
`getContent()`	Object	Converts the data to a Java object, depending on its type
`getContent(Class[] types)`	Object	Same, with the types you would prefer

SourceViewer.java

```
import java.io.*;
import java.net.*;

// 3.2 Retrieving data from a URL with openStream().
public class SourceViewer {
    public static void main(String[] args) {
        String address = args.length > 0 ? args[0] : "http://example.com/";
        try {
            URL u = new URL(address);
            try (InputStream in = new BufferedInputStream(u.openStream())) {
                Reader reader = new InputStreamReader(in, "UTF-8");
                int c;
                while ((c = reader.read()) != -1) {
                    System.out.print((char) c);
                }
            }
        } catch (MalformedURLException e) {
            System.out.println(address + " is not a valid URL");
        } catch (IOException e) {
            System.out.println("Could not read " + address + ": " + e);
        }
    }
}
```

Sample output (shortened)

```
<!doctype html><html lang=en><head><meta charset=utf-8>...<title>Example Domain</title>...
```

- The two nested tries are normal: the inner one closes the stream, the outer one handles the exceptions.
- `openStream()` returns only the body, with no HTTP headers.
- Network errors throw IOException, and a bad URL throws MalformedURLException, which is a subclass of IOException. Catch the subclass first.

ContentGetter.java

```
import java.io.*;
import java.net.*;

// 3.2 Retrieving data from a URL with getContent().
public class ContentGetter {
    public static void main(String[] args) {
        String address = args.length > 0 ? args[0] : "http://example.com/";
        try {
            URL u = new URL(address);
            Object content = u.getContent();              // type depends on the data
            System.out.println("I got a " + content.getClass().getName());

            // ask for specific types, in order of preference
            Class<?>[] types = {String.class, InputStream.class};
            Object preferred = u.getContent(types);
```

```
            if (preferred instanceof String) {
                System.out.println("It is a String");
            } else if (preferred instanceof InputStream) {
                System.out.println("It is an InputStream");
            } else {
                System.out.println("None of the requested types");
            }
        } catch (MalformedURLException e) {
            System.out.println(address + " is not a valid URL");
        } catch (IOException e) {
            System.out.println("Could not read " + address + ": " + e);
        }
    }
}
```

Sample output

```
I got a sun.net.www.protocol.http.HttpURLConnection$HttpInputStream
It is an InputStream
```

`getContent()` is rarely useful: for HTML and text it just returns an InputStream. Check the result with `instanceof`.

B4. Splitting a URL into pieces (3.2)

Getter	Returns for the sample URL	Notes
`getProtocol()`	https	
`getUserInfo()`	admin	null if none
`getHost()`	www.example.com	
`getPort()`	8443	**-1** if not written in the URL
`getDefaultPort()`	443	Default for the protocol
`getAuthority()`	admin@www.example.com:8443	
`getFile()`	/docs/guide.html?lang=en&v=2	Path **plus query**
`getPath()`	/docs/guide.html	Path only
`getQuery()`	lang=en&v=2	null if none
`getRef()`	intro	The fragment; null if none

URLSplitter.java

```
import java.net.*;

// 3.2 Splitting a URL into pieces with the getter methods.
public class URLSplitter {
    public static void main(String[] args) {
        String address = args.length > 0 ? args[0]
                : "https://admin@www.example.com:8443/docs/guide.html?lang=en&v=2#intro";
        try {
            URL u = new URL(address);
            System.out.println("URL          : " + u);
            System.out.println("Protocol     : " + u.getProtocol());
            System.out.println("User info    : " + u.getUserInfo());
            System.out.println("Host         : " + u.getHost());
            System.out.println("Port         : " + u.getPort());          // -1 if not in the URL
            System.out.println("Default port : " + u.getDefaultPort());
            System.out.println("Authority    : " + u.getAuthority());
            System.out.println("File         : " + u.getFile());          // path + query
            System.out.println("Path         : " + u.getPath());
            System.out.println("Query        : " + u.getQuery());
```

```
            System.out.println("Ref          : " + u.getRef());           // the #fragment
        } catch (MalformedURLException e) {
            System.out.println(address + " is not a valid URL");
        }
    }
}
```

Sample output

```
URL          : https://admin@www.example.com:8443/docs/guide.html?lang=en&v=2#intro
Protocol     : https
User info    : admin
Host         : www.example.com
Port         : 8443
Default port : 443
Authority    : admin@www.example.com:8443
File         : /docs/guide.html?lang=en&v=2
Path         : /docs/guide.html
Query        : lang=en&v=2
Ref          : intro
```

B5. Equality, comparison and conversion (3.2)

Method	Behaviour
`equals(Object)`	Same protocol, host (**resolved via DNS**), port, file and fragment. It is a blocking network call.
`hashCode()`	Also resolves the host, so URLs are poor keys in a HashMap or HashSet.
`sameFile(URL)`	Like equals, but ignores the fragment.
`toString()`, `toExternalForm()`	The URL as text. The two give the same result.
`toURI()`	Converts to a URI. Throws URISyntaxException if the URL is not a valid URI.

URLEquality.java

```
import java.net.*;

// 3.2 Equality, comparison and conversion of URLs.
public class URLEquality {
    public static void main(String[] args) {
        try {
            URL a = new URL("http://localhost/index.html");
            URL b = new URL("http://127.0.0.1/index.html");
            URL c = new URL("http://www.example.com/page.html#first");
            URL d = new URL("http://www.example.com/page.html#second");

            // equals() resolves host names, so this is a blocking DNS operation
            System.out.println("localhost equals 127.0.0.1 : " + a.equals(b));

            // the fragment counts for equals() but not for sameFile()
            System.out.println("c.equals(d)                : " + c.equals(d));
            System.out.println("c.sameFile(d)              : " + c.sameFile(d));

            // conversion
            System.out.println("toString()                 : " + c.toString());
            System.out.println("toExternalForm()           : " + c.toExternalForm());
            URI uri = c.toURI();
            System.out.println("toURI()                    : " + uri);
        } catch (MalformedURLException e) {
            System.out.println("Malformed URL: " + e.getMessage());
        } catch (URISyntaxException e) {
            System.out.println("Cannot convert to URI: " + e.getMessage());
        }
```

```
    }
}
```

Sample output

```
localhost equals 127.0.0.1 : true
c.equals(d)                : false
c.sameFile(d)              : true
toString()                 : http://www.example.com/page.html#first
toExternalForm()           : http://www.example.com/page.html#first
toURI()                    : http://www.example.com/page.html#first
```

localhost and 127.0.0.1 are different text but equal URLs, because `equals()` compared the resolved addresses.

B6. The URI class: constructing a URI (3.3)

Basis	URL	URI
Purpose	Identify **and retrieve** a resource	Identify and manipulate only; it cannot fetch
Class	final, immutable	final, immutable
Protocols	Only those with a handler	Any scheme, even invented ones
Relative form	Must be resolved at construction	Can stay relative
equals()	Uses DNS, blocking	Compares text, no network
Conformance	Loose	Strict RFC 2396 syntax
Convert	`url.toURI()`	`uri.toURL()`

Constructor	Encodes illegal characters?
`URI(String str)`	No. Illegal characters throw URISyntaxException
`URI(scheme, schemeSpecificPart, fragment)`	Yes
`URI(scheme, host, path, fragment)`	Yes
`URI(scheme, authority, path, query, fragment)`	Yes
`URI(scheme, userInfo, host, port, path, query, fragment)`	Yes
`URI.create(String str)`	No. A static method that throws the unchecked IllegalArgumentException

URIConstructors.java

```
import java.net.*;

// 3.3 Constructing a URI.
public class URIConstructors {
    public static void main(String[] args) {
        try {
            // from one string
            System.out.println(new URI("http://www.example.com/docs/index.html?lang=en#top"));

            // scheme, scheme-specific part, fragment
            System.out.println(new URI("tel", "+977-1-4331000", null));

            // scheme, host, path, fragment: illegal characters are escaped for you
            System.out.println(new URI("http", "www.example.com", "/a b/c.html", "top"));

            // scheme, authority, path, query, fragment
```

```
            System.out.println(new URI("http", "www.example.com:8080", "/docs", "lang=en", "top"));

            // scheme, userInfo, host, port, path, query, fragment
            System.out.println(new URI("https", "admin", "www.example.com", 8443,
                                       "/docs", "lang=en", "top"));

            // URI.create() throws an unchecked exception instead
            System.out.println(URI.create("urn:isbn:9781449357672"));

            // a URI need not be absolute, and the scheme need not be known to Java
            System.out.println(new URI("docs/index.html"));
            System.out.println(new URI("myscheme://anything/at/all"));

            // the one-string constructor does not escape: a space is illegal
            System.out.println(new URI("http://www.example.com/a b"));
        } catch (URISyntaxException e) {
            System.out.println("Bad URI: " + e.getMessage());
        }
    }
}
```

Sample output

```
http://www.example.com/docs/index.html?lang=en#top
tel:+977-1-4331000
http://www.example.com/a%20b/c.html#top
http://www.example.com:8080/docs?lang=en#top
https://admin@www.example.com:8443/docs?lang=en#top
urn:isbn:9781449357672
docs/index.html
myscheme://anything/at/all
Bad URI: Illegal character in path at index 24: http://www.example.com/a b
```

B7. The parts of the URI (3.3)

Method	Returns
`getScheme()`	Scheme, null for a relative URI
`getSchemeSpecificPart()` / `getRawSchemeSpecificPart()`	Everything between the scheme and the fragment
`getAuthority()`, `getUserInfo()`, `getHost()`	Null for opaque URIs
`getPort()`	-1 if there is none
`getPath()` / `getRawPath()`	Decoded / still encoded
`getQuery()` / `getRawQuery()`	Decoded / still encoded
`getFragment()` / `getRawFragment()`	Decoded / still encoded
`isAbsolute()`	True if it has a scheme
`isOpaque()`	True for URIs such as mailto: and tel:

URISplitter.java

```
import java.net.*;

// 3.3 The parts of the URI.
public class URISplitter {
    public static void main(String[] args) {
        String[] addresses = args.length > 0 ? args : new String[] {
            "https://admin@www.example.com:8443/a%20b/c.html?lang=en#top",
            "tel:+977-1-4331000",
            "docs/index.html"
        };
```

```
        for (String address : addresses) {
            try {
                URI u = new URI(address);
                System.out.println("URI: " + u);
                System.out.println("  absolute?   " + u.isAbsolute());
                System.out.println("  opaque?     " + u.isOpaque());
                System.out.println("  scheme      " + u.getScheme());
                if (u.isOpaque()) {
                    System.out.println("  specific    " + u.getSchemeSpecificPart());
                } else {
                    System.out.println("  authority   " + u.getAuthority());
                    System.out.println("  user info   " + u.getUserInfo());
                    System.out.println("  host        " + u.getHost());
                    System.out.println("  port        " + u.getPort());
                    System.out.println("  path        " + u.getPath());       // decoded
                    System.out.println("  raw path    " + u.getRawPath());    // still encoded
                    System.out.println("  query       " + u.getQuery());
                }
                System.out.println("  fragment    " + u.getFragment());
            } catch (URISyntaxException e) {
                System.out.println(address + " is not a valid URI");
            }
        }
    }
}
```

Sample output (first two URIs)

```
URI: https://admin@www.example.com:8443/a%20b/c.html?lang=en#top
  absolute?   true
  opaque?     false
  scheme      https
  authority   admin@www.example.com:8443
  user info   admin
  host        www.example.com
  port        8443
  path        /a b/c.html
  raw path    /a%20b/c.html
  query       lang=en
  fragment    top
URI: tel:+977-1-4331000
  absolute?   true
  opaque?     true
  scheme      tel
  specific    +977-1-4331000
  fragment    null
```

B8. Resolving relative URIs (3.3)

Method	Direction	Example
`base.resolve(uri)`	Relative to absolute	chapter2.html becomes http://.../docs/intro/chapter2.html
`base.relativize(uri)`	Absolute to relative	http://.../docs/intro/chapter1.html becomes intro/chapter1.html
`uri.normalize()`	Removes . and ..	/a/./b/../c.html becomes /a/c.html

URIResolver.java

```
import java.net.*;

// 3.3 Resolving relative URIs: resolve(), relativize() and normalize().
public class URIResolver {
    public static void main(String[] args) {
        try {
            URI base = new URI("http://www.example.com/docs/intro/index.html");

            // relative -> absolute
            System.out.println(base.resolve("chapter2.html"));
            System.out.println(base.resolve("../images/logo.png"));
            System.out.println(base.resolve("/about.html"));
            System.out.println(base.resolve(new URI("https://other.example.org/x")));

            // absolute -> relative
            URI top = new URI("http://www.example.com/docs/");
            URI page = new URI("http://www.example.com/docs/intro/chapter1.html");
            System.out.println(top.relativize(page));

            // remove . and .. from the path
            URI messy = new URI("http://www.example.com/a/./b/../c.html");
            System.out.println(messy.normalize());
        } catch (URISyntaxException e) {
            System.out.println("Bad URI: " + e.getMessage());
        }
    }
}
```

Sample output

```
http://www.example.com/docs/intro/chapter2.html
http://www.example.com/docs/images/logo.png
http://www.example.com/about.html
https://other.example.org/x
intro/chapter1.html
http://www.example.com/a/c.html
```

B9. Equality, comparison and string representation (3.3)

Method	Behaviour
`equals(Object)`	Scheme and host ignore case; hex digits in %XX ignore case; everything else is exact. **No DNS.**
`hashCode()`	Consistent with equals, so URIs are safe in a HashMap.
`compareTo(URI)`	URI implements Comparable, so URIs can be sorted.
`toString()`	The URI as text. May contain non-ASCII characters.
`toASCIIString()`	The URI as text with non-ASCII characters percent-encoded.

URIEquality.java

```
import java.net.*;

// 3.3 Equality, comparison and string representation of URIs.
public class URIEquality {
    public static void main(String[] args) {
        try {
            URI a = new URI("HTTP://WWW.Example.COM/index.html");
            URI b = new URI("http://www.example.com/index.html");
            URI c = new URI("http://www.example.com/INDEX.html");

            // scheme and host ignore case, the path does not; no DNS lookup happens
```

```
            System.out.println("a.equals(b) : " + a.equals(b));
            System.out.println("b.equals(c) : " + b.equals(c));
            System.out.println("a.hashCode() == b.hashCode() : " + (a.hashCode() == b.hashCode()));

            // compareTo() gives an ordering, so URIs can be sorted
            System.out.println("b.compareTo(c) > 0 : " + (b.compareTo(c) > 0));

            // toString() may hold non-ASCII text, toASCIIString() never does
            URI cafe = new URI("http", "www.example.com", "/caf\u00e9", null);
            System.out.println("toASCIIString() : " + cafe.toASCIIString());
        } catch (URISyntaxException e) {
            System.out.println("Bad URI: " + e.getMessage());
        }
    }
}
```

Sample output

```
a.equals(b) : true
b.equals(c) : false
a.hashCode() == b.hashCode() : true
b.compareTo(c) > 0 : true
toASCIIString() : http://www.example.com/caf%C3%A9
```

B10. URLEncoder and URLDecoder (3.4)

Method	Effect
`URLEncoder.encode(String s, String charset)`	Letters, digits and `- _ . *` stay the same, a space becomes `+`, and everything else becomes %XX.
`URLDecoder.decode(String s, String charset)`	Reverses it. + becomes a space, and %XX becomes the original character.

Always pass `"UTF-8"`. The version without a charset is deprecated. Both methods throw the checked `UnsupportedEncodingException`.

EncoderDecoder.java

```
import java.io.*;
import java.net.*;

// 3.4 x-www-form-urlencoded: URLEncoder and URLDecoder.
public class EncoderDecoder {
    public static void main(String[] args) {
        String[] samples = {"This has spaces", "a+b=c", "50%", "x&y", "/path", "*.~-_", "caf\u00e9"};
        try {
            for (String s : samples) {
                String encoded = URLEncoder.encode(s, "UTF-8");
                String decoded = URLDecoder.decode(encoded, "UTF-8");
                System.out.println(encoded + "   decodes back: " + decoded.equals(s));
            }

            // encode only the values, never the whole URL
            String wrong = URLEncoder.encode("http://www.example.com/s?q=a b", "UTF-8");
            String right = "http://www.example.com/s?q=" + URLEncoder.encode("a b", "UTF-8");
            System.out.println("Wrong: " + wrong);
            System.out.println("Right: " + right);
        } catch (UnsupportedEncodingException e) {
            throw new RuntimeException("This JVM does not support UTF-8");
        }
    }
}
```

Sample output

```
This+has+spaces   decodes back: true
a%2Bb%3Dc   decodes back: true
50%25   decodes back: true
x%26y   decodes back: true
%2Fpath   decodes back: true
*.%7E-_   decodes back: true
caf%C3%A9   decodes back: true
Wrong: http%3A%2F%2Fwww.example.com%2Fs%3Fq%3Da+b
Right: http://www.example.com/s?q=a+b
```

The QueryString class

The textbook wraps URLEncoder in a small class that builds a query one pair at a time. It is reused in B13.

QueryString.java

```
import java.io.*;
import java.net.*;

// 3.4 Builds an x-www-form-urlencoded query string one pair at a time.
public class QueryString {
    private StringBuilder query = new StringBuilder();

    public synchronized void add(String name, String value) {
        if (query.length() > 0) {
            query.append('&');
        }
        try {
            query.append(URLEncoder.encode(name, "UTF-8"));
            query.append('=');
            query.append(URLEncoder.encode(value, "UTF-8"));
        } catch (UnsupportedEncodingException e) {
            throw new RuntimeException("This JVM does not support UTF-8");
        }
    }

    public synchronized String getQuery() {
        return query.toString();
    }

    public String toString() {
        return getQuery();
    }

    public static void main(String[] args) {
        QueryString qs = new QueryString();
        qs.add("name", "Hari Bahadur");
        qs.add("course", "Network Programming");
        qs.add("marks", "60+20");
        System.out.println(qs);
    }
}
```

Sample output

```
name=Hari+Bahadur&course=Network+Programming&marks=60%2B20
```

B11. Proxies: system properties and the Proxy class (3.5)

System property	Meaning
`http.proxyHost`, `http.proxyPort`	HTTP proxy for every connection in the JVM
`https.proxyHost`, `https.proxyPort`	The same for HTTPS

System property	Meaning
`http.nonProxyHosts`	Hosts to reach directly, separated by |, with * allowed, e.g. *.example.com|localhost
`socksProxyHost`, `socksProxyPort`	SOCKS proxy

The `Proxy` class allows a **different proxy for each connection**: create it with a type and a `SocketAddress`, then pass it to `openConnection(proxy)`.

`ProxyDemo.java`

```
import java.io.*;
import java.net.*;

// 3.5 Proxies: system properties and the Proxy class.
// Start LocalTestServer first: it also behaves as an HTTP proxy on port 8000.
public class ProxyDemo {
    public static void main(String[] args) {
        try {
            URL u = new URL("http://www.example.com/page.html");

            // 1. system properties: apply to every connection in this JVM
            System.setProperty("http.proxyHost", "localhost");
            System.setProperty("http.proxyPort", "8000");
            System.setProperty("http.nonProxyHosts", "*.intranet.example.com");
            System.out.println("System properties: " + firstLine(u.openConnection()));
            System.clearProperty("http.proxyHost");
            System.clearProperty("http.proxyPort");
            System.clearProperty("http.nonProxyHosts");

            // 2. the Proxy class: applies to this one connection only
            SocketAddress address = new InetSocketAddress("localhost", 8000);
            Proxy proxy = new Proxy(Proxy.Type.HTTP, address);
            System.out.println("Proxy object: " + proxy);
            System.out.println("Proxy class: " + firstLine(u.openConnection(proxy)));

            // 3. Proxy.NO_PROXY: go directly, ignoring any proxy settings
            System.out.println("Proxy type of NO_PROXY: " + Proxy.NO_PROXY.type());
        } catch (IOException e) {
            System.out.println("Error (is LocalTestServer running?): " + e);
        }
    }

    private static String firstLine(URLConnection connection) throws IOException {
        try (BufferedReader in = new BufferedReader(
                new InputStreamReader(connection.getInputStream()))) {
            return in.readLine();
        }
    }
}
```

Sample output (LocalTestServer running)

```
System properties: Answered by the proxy for http://www.example.com/page.html
Proxy object: HTTP @ localhost/127.0.0.1:8000
Proxy class: Answered by the proxy for http://www.example.com/page.html
Proxy type of NO_PROXY: DIRECT
```

www.example.com was never contacted. The request went to the proxy on port 8000, which saw the full URL.

B12. The ProxySelector class (3.5)

`ProxySelector` is an abstract class. Subclass it to decide **per URI** which proxy to use, then install it with `ProxySelector.setDefault()`.

Method	Purpose
`List<Proxy> select(URI uri)`	Returns the proxies to try, in order. Use Proxy.NO_PROXY to connect directly.
`void connectFailed(URI, SocketAddress, IOException)`	Called when a proxy could not be reached, so you can stop using it.
`static setDefault(ProxySelector)`	Installs the selector for the whole JVM.

`LocalProxySelector.java`

```
import java.io.*;
import java.net.*;
import java.util.*;

// 3.5 The ProxySelector class: choose a proxy for each URI.
// Start LocalTestServer first: it also behaves as an HTTP proxy on port 8000.
public class LocalProxySelector extends ProxySelector {
    private List<URI> failed = new ArrayList<>();
    private Proxy proxy = new Proxy(Proxy.Type.HTTP, new InetSocketAddress("localhost", 8000));

    public List<Proxy> select(URI uri) {
        List<Proxy> result = new ArrayList<>();
        if (failed.contains(uri) || !"http".equalsIgnoreCase(uri.getScheme())
                || uri.getHost().endsWith(".intranet.example.com")) {
            result.add(Proxy.NO_PROXY);       // go direct
        } else {
            result.add(proxy);
        }
        return result;
    }

    public void connectFailed(URI uri, SocketAddress address, IOException e) {
        failed.add(uri);                      // do not use the proxy for this URI again
    }

    public static void main(String[] args) throws IOException {
        LocalProxySelector selector = new LocalProxySelector();
        ProxySelector.setDefault(selector);   // every URL in this JVM now asks the selector

        String[] tests = {"http://www.example.com/", "https://www.example.com/",
                          "http://wiki.intranet.example.com/"};
        for (String test : tests) {
            System.out.println(test + " -> " + selector.select(URI.create(test)));
        }

        URL u = new URL("http://www.example.com/page.html");
        try (BufferedReader in = new BufferedReader(new InputStreamReader(u.openStream()))) {
            System.out.println(in.readLine());
        }
    }
}
```

Sample output (LocalTestServer running)

```
http://www.example.com/ -> [HTTP @ localhost/127.0.0.1:8000]
https://www.example.com/ -> [DIRECT]
http://wiki.intranet.example.com/ -> [DIRECT]
Answered by the proxy for http://www.example.com/page.html
```

B13. Communicating with server-side programs through GET (3.6)

Following the steps in A5: build the query with QueryString, append it to the address, and read the response. Start the test server first.

FormGet.java

```
import java.io.*;
import java.net.*;

// 3.6 Communicating with a server-side program through GET.
// Start LocalTestServer first, or pass the address of another GET form.
public class FormGet {
    public static void main(String[] args) {
        String target = args.length > 0 ? args[0] : "http://localhost:8000/search";

        QueryString query = new QueryString();
        query.add("q", "java network programming");
        query.add("lang", "en");

        try {
            URL u = new URL(target + "?" + query);
            System.out.println("GET " + u);
            try (BufferedReader in = new BufferedReader(
                    new InputStreamReader(u.openStream(), "UTF-8"))) {
                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println(line);
                }
            }
        } catch (MalformedURLException e) {
            System.out.println(target + " is not a valid URL");
        } catch (IOException e) {
            System.out.println("Could not read the response: " + e);
        }
    }
}
```

Sample output (LocalTestServer running)

```
GET http://localhost:8000/search?q=java+network+programming&lang=en
Search received:
  q = java network programming
  lang = en
```

The test server (not for the exam)

This uses the JDK's built-in `com.sun.net.httpserver` package. It is only here so the programs have something to talk to. Run it with `java LocalTestServer`.

LocalTestServer.java

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.*;
import java.util.*;

// A small server on port 8000 for trying out the programs in sections 3.5 - 3.7.
//   /search?q=...  echoes the decoded query parameters
//   /secret        needs HTTP Basic login: student / network123
//   any absolute-URI request is answered as if this server were an HTTP proxy
public class LocalTestServer {
    private static final String LOGIN = "student:network123";

    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);
        server.createContext("/", LocalTestServer::root);
        server.createContext("/search", LocalTestServer::search);
```

```
        server.createContext("/secret", LocalTestServer::secret);
        server.start();
        System.out.println("LocalTestServer running on http://localhost:8000/");
    }

    private static void root(HttpExchange ex) throws IOException {
        URI uri = ex.getRequestURI();
        if (uri.isAbsolute()) {
            send(ex, 200, "Answered by the proxy for " + uri + "\n");
        } else {
            send(ex, 200, "Hello from LocalTestServer, you asked for " + uri + "\n");
        }
    }

    private static void search(HttpExchange ex) throws IOException {
        StringBuilder reply = new StringBuilder("Search received:\n");
        String query = ex.getRequestURI().getRawQuery();
        if (query != null) {
            for (String pair : query.split("&")) {
                String[] parts = pair.split("=", 2);
                reply.append("  ").append(URLDecoder.decode(parts[0], "UTF-8")).append(" = ")
                     .append(parts.length > 1 ? URLDecoder.decode(parts[1], "UTF-8") : "")
                     .append("\n");
            }
        }
        send(ex, 200, reply.toString());
    }

    private static void secret(HttpExchange ex) throws IOException {
        String expected = "Basic " + Base64.getEncoder().encodeToString(LOGIN.getBytes("UTF-8"));
        if (expected.equals(ex.getRequestHeaders().getFirst("Authorization"))) {
            send(ex, 200, "Welcome! This page is password protected.\n");
        } else {
            ex.getResponseHeaders().set("WWW-Authenticate", "Basic realm=\"Network Programming\"");
            send(ex, 401, "Login required\n");
        }
    }

    private static void send(HttpExchange ex, int status, String body) throws IOException {
        byte[] bytes = body.getBytes("UTF-8");
        ex.getResponseHeaders().set("Content-Type", "text/plain; charset=utf-8");
        ex.sendResponseHeaders(status, bytes.length);
        try (OutputStream out = ex.getResponseBody()) {
            out.write(bytes);
        }
    }
}
```

B14. Authenticator and PasswordAuthentication (3.7)

- `java.net.Authenticator` is an **abstract** class. Subclass it and override `getPasswordAuthentication()`.
- Install it with the static `Authenticator.setDefault(...)`. Java then calls it automatically when a server answers 401.
- Return a `PasswordAuthentication`, or **null** to give up.
- `PasswordAuthentication` is a final class with one constructor, `(String userName, char[] password)`, and the getters `getUserName()` and `getPassword()`.

Protected getter	Returns
`getRequestingHost()`	Host name asking for the login
`getRequestingSite()`	The same as an InetAddress

Protected getter	Returns
`getRequestingPort()`	Port
`getRequestingProtocol()`	Protocol, e.g. http
`getRequestingPrompt()`	The realm, e.g. Network Programming
`getRequestingScheme()`	Authentication scheme, e.g. basic
`getRequestingURL()`	Full URL requested
`getRequestorType()`	SERVER or PROXY

SecureViewer.java

```
import java.io.*;
import java.net.*;

// 3.7 Accessing a password-protected site: Authenticator and PasswordAuthentication.
// Start LocalTestServer first (login: student / network123).
public class SecureViewer {
    public static void main(String[] args) {
        String user = args.length > 0 ? args[0] : "student";
        String password = args.length > 1 ? args[1] : "network123";
        Authenticator.setDefault(new SimpleAuthenticator(user, password));

        try {
            URL u = new URL("http://localhost:8000/secret");
            try (BufferedReader in = new BufferedReader(new InputStreamReader(u.openStream()))) {
                System.out.println(in.readLine());
            }
        } catch (IOException e) {
            System.out.println("Could not read the page: " + e);
        }
    }
}

class SimpleAuthenticator extends Authenticator {
    private String user;
    private char[] password;

    SimpleAuthenticator(String user, String password) {
        this.user = user;
        this.password = password.toCharArray();
    }

    // Java calls this only when the server answers 401 Unauthorized
    protected PasswordAuthentication getPasswordAuthentication() {
        System.out.println("Host   : " + getRequestingHost());
        System.out.println("Port   : " + getRequestingPort());
        System.out.println("Scheme : " + getRequestingScheme());
        System.out.println("Prompt : " + getRequestingPrompt());
        System.out.println("URL    : " + getRequestingURL());
        return new PasswordAuthentication(user, password);
    }
}
```

Sample output (LocalTestServer running)

```
Host   : localhost
Port   : 8000
Scheme : basic
Prompt : Network Programming
URL    : http://localhost:8000/secret
Welcome! This page is password protected.
```

- The program never mentions 401 or Base64: Java handles the whole flow in A6.

- If the password is wrong, Java keeps asking the Authenticator. With a fixed password it finally gave up with `ProtocolException: Server redirected too many times (20)`.

B15. The JPasswordField class (3.7)

`javax.swing.JPasswordField` is a text field that shows dots instead of the typed letters, which makes it right for an Authenticator that asks the user.

Method	Purpose
`JPasswordField(int columns)`	Constructor
`char[] getPassword()`	Returns the text as a char array (use this one)
`getText()`	Deprecated for passwords
`setEchoChar(char c)`, `getEchoChar()`	The character shown for each letter

Why char[] instead of String? Strings are immutable and stay in memory until garbage collected. A char array can be overwritten as soon as you are done, as the program does with `Arrays.fill`.

`DialogAuthenticator.java`

```
import java.awt.*;
import java.io.*;
import java.net.*;
import java.util.Arrays;
import javax.swing.*;

// 3.7 Asking the user for a password with JPasswordField.
// Start LocalTestServer first (login: student / network123).
public class DialogAuthenticator extends Authenticator {

    protected PasswordAuthentication getPasswordAuthentication() {
        JTextField userField = new JTextField(15);
        JPasswordField passwordField = new JPasswordField(15);   // shows dots, not letters

        JPanel panel = new JPanel(new GridLayout(2, 2, 5, 5));
        panel.add(new JLabel("User name:"));
        panel.add(userField);
        panel.add(new JLabel("Password:"));
        panel.add(passwordField);

        int choice = JOptionPane.showConfirmDialog(null, panel,
                "Login: " + getRequestingPrompt(),
                JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);
        if (choice != JOptionPane.OK_OPTION) {
            return null;                                          // user cancelled
        }

        char[] password = passwordField.getPassword();            // char[], not String
        PasswordAuthentication result = new PasswordAuthentication(userField.getText(), password);
        Arrays.fill(password, ' ');                               // wipe our copy
        return result;
    }

    public static void main(String[] args) {
        Authenticator.setDefault(new DialogAuthenticator());
        try {
            URL u = new URL("http://localhost:8000/secret");
            try (BufferedReader in = new BufferedReader(new InputStreamReader(u.openStream()))) {
                System.out.println(in.readLine());
            }
        } catch (IOException e) {
            System.out.println("Could not read the page: " + e);
        }
        System.exit(0);                                           // Swing keeps the JVM alive
```

```
    }
}
```

Sample output (after typing student / network123 in the dialog)

```
Welcome! This page is password protected.
```

`System.exit(0)` is needed because Swing's event thread would otherwise keep the program running.

B16. Tips for Unit 3 in the exam

- Learn the difference tables: URI vs URL vs URN, URL vs URI classes, `getFile()` vs `getPath()`, `equals()` vs `sameFile()`.
- Always handle the exception: `MalformedURLException` for URL, `URISyntaxException` for URI, `UnsupportedEncodingException` for the encoder, `IOException` for reading.
- Most likely programs: split a URL into parts (B4), read a page (B3), resolve relative URIs (B8), encode/decode (B10), GET with a query (B13), Authenticator (B14).
- For a proxy question, write the system properties first, then the Proxy class. Add ProxySelector if it asks for one.
- Remember: the fragment is never sent to the server, `getPort()` returns -1 when absent, and URLEncoder turns a space into +.

Part C: Go (job track)

C1. The net/url package

Go has **one type, url.URL**, for both URLs and URIs. It is a plain struct whose fields you read and set directly. Fetching is a separate job, done by the `net/http` package.

Java	Go
URL and URI classes	`url.URL` struct, in net/url
Immutable	Mutable fields (copy the struct before changing a shared one)
Fetch with `url.openStream()`	`http.Get(address)` in net/http
`MalformedURLException`, `URISyntaxException`	An `error` return value
URLEncoder / URLDecoder	`url.QueryEscape` / `url.QueryUnescape`
QueryString (write your own)	`url.Values` (built in)
Proxy / ProxySelector	`http.Transport.Proxy` field (a function)
Authenticator callback	`req.SetBasicAuth(user, password)`

C2. Creating URLs (Java B2)

`go/url_build/main.go`

```
package main

import (
    "fmt"
    "log"
    "net"
    "net/url"
)

// 3.2 Creating new URLs (Java: the URL constructors).
func main() {
    // 1. from one complete string (Java: new URL(spec))
    u1, err := url.Parse("http://www.example.com:8080/docs/index.html?lang=en#top")
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("url.Parse           :", u1)

    // 2. from parts (Java: new URL(protocol, host, file))
    u2 := &url.URL{Scheme: "http", Host: "www.example.com", Path: "/docs/index.html"}
    fmt.Println("url.URL{...}        :", u2)

    // 3. with a port (Java: new URL(protocol, host, port, file))
    u3 := &url.URL{
        Scheme: "https",
        Host:   net.JoinHostPort("www.example.com", "8443"),
        Path:   "/docs/index.html",
    }
    fmt.Println("url.URL{... port}   :", u3)

    // 4. relative to a base URL (Java: new URL(base, relative))
    base, _ := url.Parse("http://www.example.com/docs/intro/index.html")
    for _, relative := range []string{"chapter1.html", "../img/logo.png", "/about.html"} {
        u, err := base.Parse(relative)
        if err != nil {
            log.Fatal(err)
```

```
        }
        fmt.Printf("base.Parse(%-16q): %s\n", relative, u)
    }

    // url.Parse only checks syntax: an unknown scheme is fine, a bad host is not
    unknown, err := url.Parse("htp://www.example.com/")
    fmt.Println("unknown scheme      :", unknown, "error:", err)
    _, err = url.Parse("http://bad host/")
    fmt.Println("bad host            :", err)
}
```

Sample output

```
url.Parse           : http://www.example.com:8080/docs/index.html?lang=en#top
url.URL{...}        : http://www.example.com/docs/index.html
url.URL{... port}   : https://www.example.com:8443/docs/index.html
base.Parse("chapter1.html" ): http://www.example.com/docs/intro/chapter1.html
base.Parse("../img/logo.png"): http://www.example.com/docs/img/logo.png
base.Parse("/about.html"   ): http://www.example.com/about.html
unknown scheme      : htp://www.example.com/ error: <nil>
bad host            : parse "http://bad host/": invalid character " " in host name
```

Difference from Java: `url.Parse` only checks syntax, so an unknown scheme like `htp` is accepted. Java rejected it because it had no handler.

Which schemes can the HTTP client fetch?

go/protocol_tester/main.go

```
package main

import (
    "fmt"
    "net/http"
    "strings"
    "time"
)

// 3.2 Which schemes can Go's HTTP client fetch? (Java: ProtocolTester)
func main() {
    client := &http.Client{Timeout: 3 * time.Second}
    for _, scheme := range []string{"http", "https", "ftp", "file", "mailto", "telnet", "gopher"} {
        // nothing listens on port 1, so a supported scheme fails only when connecting
        _, err := client.Get(scheme + "://localhost:1/")
        if err != nil && strings.Contains(err.Error(), "unsupported protocol scheme") {
            fmt.Println(scheme, "is NOT supported")
        } else {
            fmt.Println(scheme, "is supported")
        }
    }
}
```

Sample output

```
http is supported
https is supported
ftp is NOT supported
file is NOT supported
mailto is NOT supported
telnet is NOT supported
gopher is NOT supported
```

C3. Retrieving data (Java B3)

go/source_viewer/main.go

```
package main

import (
    "fmt"
    "io"
    "log"
    "net/http"
    "os"
)

// 3.2 Retrieving data from a URL (Java: SourceViewer with openStream).
func main() {
    address := "http://example.com/"
    if len(os.Args) > 1 {
        address = os.Args[1]
    }

    resp, err := http.Get(address)
    if err != nil {
        log.Fatal(err)
    }
    defer resp.Body.Close() // Body is an io.ReadCloser, like Java's InputStream

    if resp.StatusCode != http.StatusOK {
        fmt.Println("Server answered:", resp.Status)
    }
    io.Copy(os.Stdout, resp.Body) // streams the body without loading it all into memory
}
```

- **Always close the body** with `defer resp.Body.Close()`, or the connection cannot be reused.
- A 404 or 500 is **not** an error in Go: `err` is nil, so check `resp.StatusCode`. Java's openStream throws an IOException for those.
- `io.Copy` streams the body, so a huge download doesn't fill memory.

go/content_getter/main.go

```
package main

import (
    "fmt"
    "log"
    "net/http"
    "os"
)

// 3.2 What kind of data is at this URL? (Java: ContentGetter with getContent)
func main() {
    address := "http://example.com/"
    if len(os.Args) > 1 {
        address = os.Args[1]
    }

    resp, err := http.Get(address)
    if err != nil {
        log.Fatal(err)
    }
    defer resp.Body.Close()

    fmt.Println("Server says :", resp.Header.Get("Content-Type"))

    // sniff the type from the first bytes; 512 is all DetectContentType looks at
    buf := make([]byte, 512)
    n, _ := resp.Body.Read(buf)
```

```
    fmt.Println("Sniffed     :", http.DetectContentType(buf[:n]))
}
```

Sample output

```
Server says : text/html; charset=utf-8
Sniffed     : text/html; charset=utf-8
```

C4. Splitting a URL (Java B4)

go/url_split/main.go

```
package main

import (
    "fmt"
    "log"
    "net/url"
    "os"
)

// 3.2 Splitting a URL into pieces (Java: URLSplitter). The parts are struct fields.
func main() {
    address := "https://admin@www.example.com:8443/docs/guide.html?lang=en&v=2#intro"
    if len(os.Args) > 1 {
        address = os.Args[1]
    }

    u, err := url.Parse(address)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("URL          :", u)
    fmt.Println("Scheme       :", u.Scheme)
    fmt.Println("User         :", u.User)
    fmt.Println("Host         :", u.Host) // host:port together
    fmt.Println("Hostname()   :", u.Hostname())
    fmt.Println("Port()       :", u.Port()) // empty string if not in the URL
    fmt.Println("RequestURI() :", u.RequestURI())
    fmt.Println("Path         :", u.Path)
    fmt.Println("RawQuery     :", u.RawQuery)
    fmt.Println("Query()      :", u.Query()) // parsed into a map
    fmt.Println("Fragment     :", u.Fragment)
}
```

Sample output

```
URL          : https://admin@www.example.com:8443/docs/guide.html?lang=en&v=2#intro
Scheme       : https
User         : admin
Host         : www.example.com:8443
Hostname()   : www.example.com
Port()       : 8443
RequestURI() : /docs/guide.html?lang=en&v=2
Path         : /docs/guide.html
RawQuery     : lang=en&v=2
Query()      : map[lang:[en] v:[2]]
Fragment     : intro
```

Java	Go	Watch out
`getHost()`	`u.Hostname()`	`u.Host` includes the port
`getPort()` returns -1	`u.Port()` returns `""`	A string, and empty when absent
`getFile()`	`u.RequestURI()`	Path plus query

Java	Go	Watch out
`getQuery()` text	`u.RawQuery` / `u.Query()`	`Query()` parses it into a map
`getRef()`	`u.Fragment`	

C5. Equality and conversion (Java B5)

`go/url_compare/main.go`

```
package main

import (
    "fmt"
    "net/url"
)

// 3.2 Equality, comparison and conversion (Java: URLEquality).
func main() {
    a, _ := url.Parse("http://localhost/index.html")
    b, _ := url.Parse("http://127.0.0.1/index.html")
    c, _ := url.Parse("http://www.example.com/page.html#first")
    d, _ := url.Parse("http://www.example.com/page.html#second")

    // Go compares text and never touches DNS (Java's equals does a lookup)
    fmt.Println("localhost == 127.0.0.1 :", a.String() == b.String())

    fmt.Println("c == d                 :", c.String() == d.String())
    fmt.Println("sameFile(c, d)         :", sameFile(c, d))

    // conversion
    fmt.Println("String()               :", c.String())
    fmt.Println("RequestURI()           :", c.RequestURI())
}

// sameFile compares two URLs ignoring the fragment, like Java's URL.sameFile.
func sameFile(x, y *url.URL) bool {
    p, q := *x, *y // copy the structs so the originals are not changed
    p.Fragment, q.Fragment = "", ""
    return p.String() == q.String()
}
```

Sample output

```
localhost == 127.0.0.1 : false
c == d                 : false
sameFile(c, d)         : true
String()               : http://www.example.com/page.html#first
RequestURI()           : /page.html
```

Go compares text and never does DNS, so localhost and 127.0.0.1 differ here, the opposite of Java's `URL.equals`. There is no built-in `sameFile`, so the program writes one.

C6. Constructing a URI (Java B6)

`go/uri_build/main.go`

```
package main

import (
    "fmt"
    "net/url"
)

// 3.3 Constructing a URI (Java: URIConstructors). Go has one type for URLs and URIs.
func main() {
```

```
    // from one string (Java: new URI(string))
    u, _ := url.Parse("http://www.example.com/docs/index.html?lang=en#top")
    fmt.Println(u)

    // opaque: scheme + scheme-specific part (Java: new URI(scheme, ssp, fragment))
    fmt.Println(&url.URL{Scheme: "tel", Opaque: "+977-1-4331000"})

    // illegal characters in Path are escaped for you
    fmt.Println(&url.URL{Scheme: "http", Host: "www.example.com", Path: "/a b/c.html", Fragment: "top"})

    // port and query
    fmt.Println(&url.URL{Scheme: "http", Host: "www.example.com:8080", Path: "/docs", RawQuery: "lang=en", Fragment

    // user info (Java: the seven-argument constructor)
    fmt.Println(&url.URL{Scheme: "https", User: url.User("admin"), Host: "www.example.com:8443", Path: "/docs", RawQ

    // a URN and a relative reference are fine too
    urn, _ := url.Parse("urn:isbn:9781449357672")
    fmt.Println(urn)
    fmt.Println(&url.URL{Path: "docs/index.html"})

    // errors: Parse is more forgiving than Java (a space in the path is accepted)
    _, err := url.Parse("http://www.example.com/a b")
    fmt.Println("space in path, error:", err)
    _, err = url.Parse(":no-scheme")
    fmt.Println("bad URI, error:", err)
}
```

Sample output

```
http://www.example.com/docs/index.html?lang=en#top
tel:+977-1-4331000
http://www.example.com/a%20b/c.html#top
http://www.example.com:8080/docs?lang=en#top
https://admin@www.example.com:8443/docs?lang=en#top
urn:isbn:9781449357672
docs/index.html
space in path, error: <nil>
bad URI, error: parse ":no-scheme": missing protocol scheme
```

Setting fields on the struct is Go's replacement for Java's many constructors, and `Path` is escaped automatically, as with Java's multi-argument constructors. Unlike `new URI(String)`, `url.Parse` accepts a space in the path.

C7. The parts of a URI (Java B7)

go/uri_parts/main.go

```
package main

import (
    "fmt"
    "net/url"
    "os"
)

// 3.3 The parts of the URI (Java: URISplitter).
func main() {
    addresses := []string{
        "https://admin@www.example.com:8443/a%20b/c.html?lang=en#top",
        "tel:+977-1-4331000",
        "docs/index.html",
    }
    if len(os.Args) > 1 {
        addresses = os.Args[1:]
    }
```

```
    for _, address := range addresses {
        u, err := url.Parse(address)
        if err != nil {
            fmt.Println(address, "is not a valid URI:", err)
            continue
        }
        fmt.Println("URI:", u)
        fmt.Println("  IsAbs()       ", u.IsAbs())
        fmt.Println("  Scheme        ", u.Scheme)
        if u.Opaque != "" { // Java: isOpaque()
            fmt.Println("  Opaque        ", u.Opaque)
        } else {
            fmt.Println("  User          ", u.User)
            fmt.Println("  Hostname()    ", u.Hostname())
            fmt.Println("  Port()        ", u.Port())
            fmt.Println("  Path          ", u.Path)          // decoded
            fmt.Println("  EscapedPath() ", u.EscapedPath()) // still encoded
            fmt.Println("  RawQuery      ", u.RawQuery)
        }
        fmt.Println("  Fragment      ", u.Fragment)
    }
}
```

Sample output (first two URIs)

```
URI: https://admin@www.example.com:8443/a%20b/c.html?lang=en#top
  IsAbs()        true
  Scheme         https
  User           admin
  Hostname()     www.example.com
  Port()         8443
  Path           /a b/c.html
  EscapedPath()  /a%20b/c.html
  RawQuery       lang=en
  Fragment       top
URI: tel:+977-1-4331000
  IsAbs()        true
  Scheme         tel
  Opaque         +977-1-4331000
  Fragment
```

C8. Resolving relative URIs (Java B8)

go/uri_resolve/main.go

```
package main

import (
    "fmt"
    "net/url"
    "path"
    "strings"
)

// 3.3 Resolving relative URIs (Java: URIResolver).
func main() {
    base, _ := url.Parse("http://www.example.com/docs/intro/index.html")

    // relative -> absolute (Java: resolve)
    for _, r := range []string{"chapter2.html", "../images/logo.png", "/about.html", "https://other.example.org/x"} {
        ref, _ := url.Parse(r)
        fmt.Println(base.ResolveReference(ref))
    }

    // absolute -> relative (Java: relativize; Go has no built-in, so write it)
    top, _ := url.Parse("http://www.example.com/docs/")
    page, _ := url.Parse("http://www.example.com/docs/intro/chapter1.html")
```

```
    fmt.Println(relativize(top, page))

    // remove . and .. from the path (Java: normalize)
    messy, _ := url.Parse("http://www.example.com/a/./b/../c.html")
    messy.Path = path.Clean(messy.Path)
    fmt.Println(messy)
}

func relativize(base, target *url.URL) string {
    if base.Scheme == target.Scheme && base.Host == target.Host &&
        strings.HasPrefix(target.Path, base.Path) {
        return strings.TrimPrefix(target.Path, base.Path)
    }
    return target.String() // cannot be relativized: return it unchanged, as Java does
}
```

Sample output

```
http://www.example.com/docs/intro/chapter2.html
http://www.example.com/docs/images/logo.png
http://www.example.com/about.html
https://other.example.org/x
intro/chapter1.html
http://www.example.com/a/c.html
```

`ResolveReference` replaces `resolve`. Go has no `relativize`, so the program writes a small version, and `path.Clean` does the job of `normalize`.

C9. Equality and string form (Java B9)

go/uri_compare/main.go

```
package main

import (
    "fmt"
    "net/url"
    "strings"
)

// 3.3 Equality, comparison and string form (Java: URIEquality).
func main() {
    a, _ := url.Parse("HTTP://WWW.Example.COM/index.html")
    b, _ := url.Parse("http://www.example.com/index.html")
    c, _ := url.Parse("http://www.example.com/INDEX.html")

    // Parse lowercases the scheme but keeps the host's case, so write the rule yourself
    fmt.Println("a.String()   :", a.String())
    fmt.Println("equal(a, b)  :", equal(a, b))
    fmt.Println("equal(b, c)  :", equal(b, c))

    // strings.Compare gives an ordering (Java: compareTo)
    fmt.Println("b after c    :", strings.Compare(b.String(), c.String()) > 0)

    // String() percent-encodes non-ASCII text (Java: toASCIIString)
    cafe := &url.URL{Scheme: "http", Host: "www.example.com", Path: "/caf" + string(rune(233))}
    fmt.Println("String()     :", cafe.String())
}

// equal follows Java's URI.equals: scheme and host ignore case, the rest does not.
func equal(x, y *url.URL) bool {
    return strings.EqualFold(x.Scheme, y.Scheme) &&
        strings.EqualFold(x.Host, y.Host) &&
        x.EscapedPath() == y.EscapedPath() &&
        x.RawQuery == y.RawQuery &&
        x.Fragment == y.Fragment
}
```

Sample output

```
a.String()   : http://WWW.Example.COM/index.html
equal(a, b)  : true
equal(b, c)  : false
b after c    : true
String()     : http://www.example.com/caf%C3%A9
```

C10. Encoding and query strings (Java B10)

go/encoder/main.go

```
package main

import (
    "fmt"
    "log"
    "net/url"
)

// 3.4 x-www-form-urlencoded (Java: EncoderDecoder).
func main() {
    samples := []string{"This has spaces", "a+b=c", "50%", "x&y", "/path", "*.~-_", "caf" + string(rune(233))}
    for _, s := range samples {
        encoded := url.QueryEscape(s)              // Java: URLEncoder.encode
        decoded, err := url.QueryUnescape(encoded) // Java: URLDecoder.decode
        if err != nil {
            log.Fatal(err)
        }
        fmt.Printf("%s   decodes back: %v\n", encoded, decoded == s)
    }

    // a space is + in a query but %20 in a path
    fmt.Println("QueryEscape:", url.QueryEscape("a b"))
    fmt.Println("PathEscape :", url.PathEscape("a b"))

    // encode only the values, never the whole URL
    fmt.Println("Wrong:", url.QueryEscape("http://www.example.com/s?q=a b"))
    fmt.Println("Right:", "http://www.example.com/s?q="+url.QueryEscape("a b"))
}
```

Sample output

```
This+has+spaces   decodes back: true
a%2Bb%3Dc   decodes back: true
50%25   decodes back: true
x%26y   decodes back: true
%2Fpath   decodes back: true
%2A.~-_   decodes back: true
caf%C3%A9   decodes back: true
QueryEscape: a+b
PathEscape : a%20b
Wrong: http%3A%2F%2Fwww.example.com%2Fs%3Fq%3Da+b
Right: http://www.example.com/s?q=a+b
```

- Small differences from Java: Go keeps **~** and encodes *****. Java does the opposite. Both decode correctly.
- Go has two functions: `QueryEscape` (space becomes +) and `PathEscape` (space becomes %20). Using the wrong one is a common bug.

go/query_string/main.go

```
package main

import (
    "fmt"
    "log"
    "net/url"
)

// 3.4 Building and parsing a query string (Java: the QueryString class).
func main() {
    v := url.Values{} // a map[string][]string with helper methods
    v.Add("name", "Hari Bahadur")
    v.Add("course", "Network Programming")
    v.Add("marks", "60+20")

    encoded := v.Encode() // keys come out sorted
    fmt.Println(encoded)

    parsed, err := url.ParseQuery(encoded)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("marks =", parsed.Get("marks"))
}
```

Sample output

```
course=Network+Programming&marks=60%2B20&name=Hari+Bahadur
marks = 60+20
```

`url.Values` replaces the whole QueryString class. `Encode()` sorts by key, so the order differs from Java's.

C11. Proxies (Java B11)

go/proxy_client/main.go

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net/http"
    "net/url"
    "os"
    "strings"
)

// 3.5 Proxies (Java: ProxyDemo). Start testserver first: it also acts as a proxy.
func main() {
    target := "http://www.example.com/page.html"
    proxyURL, _ := url.Parse("http://localhost:8000")

    // 1. environment variables (Java: system properties); read once, on first use
    os.Setenv("HTTP_PROXY", "http://localhost:8000")
    fromEnv := &http.Client{Transport: &http.Transport{Proxy: http.ProxyFromEnvironment}}
    fmt.Println("Environment:", firstLine(fromEnv, target))

    // 2. a fixed proxy for this client only (Java: the Proxy class)
    fixed := &http.Client{Transport: &http.Transport{Proxy: http.ProxyURL(proxyURL)}}
    fmt.Println("Proxy URL  :", firstLine(fixed, target))

    // 3. Proxy: nil always connects directly (Java: Proxy.NO_PROXY)
    direct := &http.Transport{Proxy: nil}
    fmt.Println("Direct     : Proxy is nil:", direct.Proxy == nil)
```

```
}

func firstLine(client *http.Client, address string) string {
    resp, err := client.Get(address)
    if err != nil {
        log.Fatal("is testserver running? ", err)
    }
    defer resp.Body.Close()
    line, _ := bufio.NewReader(resp.Body).ReadString('\n')
    return strings.TrimSpace(line)
}
```

Sample output (testserver running)

```
Environment: Answered by the proxy for http://www.example.com/page.html
Proxy URL  : Answered by the proxy for http://www.example.com/page.html
Direct     : Proxy is nil: true
```

Java	Go
System properties http.proxyHost/Port	Environment variables HTTP_PROXY, HTTPS_PROXY, NO_PROXY
`new Proxy(Type.HTTP, address)`	`http.ProxyURL(proxyURL)`
`Proxy.NO_PROXY`	`Proxy: nil`

Note that the default HTTP client already honours the environment variables, and `ProxyFromEnvironment` never uses a proxy for localhost targets.

C12. Choosing a proxy per request (Java B12)

go/proxy_selector/main.go

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net/http"
    "net/url"
    "strings"
)

var proxyURL, _ = url.Parse("http://localhost:8000")

// choose plays the role of Java's ProxySelector.select: nil means connect directly.
func choose(r *http.Request) (*url.URL, error) {
    if r.URL.Scheme != "http" || strings.HasSuffix(r.URL.Hostname(), ".intranet.example.com") {
        return nil, nil
    }
    return proxyURL, nil
}

// 3.5 Choosing a proxy per request. Start testserver first: it also acts as a proxy.
func main() {
    for _, test := range []string{"http://www.example.com/", "https://www.example.com/", "http://wiki.intranet.examp
        req, _ := http.NewRequest("GET", test, nil)
        p, _ := choose(req)
        if p == nil {
            fmt.Println(test, "-> DIRECT")
        } else {
            fmt.Println(test, "->", p)
        }
    }
```

```
    client := &http.Client{Transport: &http.Transport{Proxy: choose}}
    resp, err := client.Get("http://www.example.com/page.html")
    if err != nil {
        log.Fatal("is testserver running? ", err)
    }
    defer resp.Body.Close()
    line, _ := bufio.NewReader(resp.Body).ReadString('\n')
    fmt.Print(line)
}
```

Sample output (testserver running)

```
http://www.example.com/ -> http://localhost:8000
https://www.example.com/ -> DIRECT
http://wiki.intranet.example.com/ -> DIRECT
Answered by the proxy for http://www.example.com/page.html
```

The whole of Java's ProxySelector subclass becomes one function assigned to `Transport.Proxy`. Returning `nil, nil` means connect directly. Go has no `connectFailed` callback.

C13. GET requests with a query (Java B13)

go/form_get/main.go

```
package main

import (
    "fmt"
    "io"
    "log"
    "net/http"
    "net/url"
    "os"
)

// 3.6 Communicating with a server-side program through GET (Java: FormGet).
// Start testserver first, or pass the address of another GET form.
func main() {
    target := "http://localhost:8000/search"
    if len(os.Args) > 1 {
        target = os.Args[1]
    }

    u, err := url.Parse(target)
    if err != nil {
        log.Fatal(err)
    }
    query := url.Values{}
    query.Set("q", "java network programming")
    query.Set("lang", "en")
    u.RawQuery = query.Encode()

    fmt.Println("GET", u)
    resp, err := http.Get(u.String())
    if err != nil {
        log.Fatal(err)
    }
    defer resp.Body.Close()
    io.Copy(os.Stdout, resp.Body)
}
```

Sample output (testserver running)

```
GET http://localhost:8000/search?lang=en&q=java+network+programming
Search received:
  lang = en
  q = java network programming
```

The test server

go/testserver/main.go

```
package main

import (
    "fmt"
    "log"
    "net/http"
    "sort"
)

// A small server on port 8000 for trying out the programs in sections 3.5 - 3.7.
//
//    /search?q=...  echoes the decoded query parameters
//    /secret        needs HTTP Basic login: student / network123
//    any absolute-URI request is answered as if this server were an HTTP proxy
func main() {
    http.HandleFunc("/", root)
    http.HandleFunc("/search", search)
    http.HandleFunc("/secret", secret)
    fmt.Println("testserver running on http://localhost:8000/")
    log.Fatal(http.ListenAndServe(":8000", nil))
}

func root(w http.ResponseWriter, r *http.Request) {
    if r.URL.IsAbs() {
        fmt.Fprintln(w, "Answered by the proxy for", r.URL)
        return
    }
    fmt.Fprintln(w, "Hello from testserver, you asked for", r.URL)
}

func search(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Search received:")
    values := r.URL.Query()
    keys := make([]string, 0, len(values))
    for k := range values {
        keys = append(keys, k)
    }
    sort.Strings(keys)
    for _, k := range keys {
        fmt.Fprintf(w, "  %s = %s\n", k, values.Get(k))
    }
}

func secret(w http.ResponseWriter, r *http.Request) {
    user, password, ok := r.BasicAuth()
    if ok && user == "student" && password == "network123" {
        fmt.Fprintln(w, "Welcome! This page is password protected.")
        return
    }
    w.Header().Set("WWW-Authenticate", `Basic realm="Network Programming"`)
    http.Error(w, "Login required", http.StatusUnauthorized)
}
```

The Java and Go test servers behave the same. The Java client programs were also run against this Go server and worked unchanged.

C14. Password-protected sites (Java B14)

go/basic_auth/main.go

```
package main

import (
    "fmt"
    "io"
    "log"
    "net/http"
    "os"
)

// 3.7 Accessing a password-protected site (Java: SecureViewer + Authenticator).
// Start testserver first (login: student / network123).
func main() {
    user, password := "student", "network123"
    if len(os.Args) > 2 {
        user, password = os.Args[1], os.Args[2]
    }
    const address = "http://localhost:8000/secret"

    // without credentials the server answers 401 and says which login it wants
    resp, err := http.Get(address)
    if err != nil {
        log.Fatal(err)
    }
    resp.Body.Close()
    fmt.Println("No login        :", resp.Status)
    fmt.Println("WWW-Authenticate:", resp.Header.Get("WWW-Authenticate"))

    // Go has no Authenticator callback: attach the login to the request yourself
    req, err := http.NewRequest("GET", address, nil)
    if err != nil {
        log.Fatal(err)
    }
    req.SetBasicAuth(user, password)

    resp, err = http.DefaultClient.Do(req)
    if err != nil {
        log.Fatal(err)
    }
    defer resp.Body.Close()
    fmt.Println("With login      :", resp.Status)
    io.Copy(os.Stdout, resp.Body)
}
```

Sample output (testserver running)

```
No login        : 401 Unauthorized
WWW-Authenticate: Basic realm="Network Programming"
With login      : 200 OK
Welcome! This page is password protected.
```

- Go has no Authenticator callback and does not retry on 401. `SetBasicAuth` sends the header on the first request.
- On the server side, `r.BasicAuth()` decodes the header for you, as the testserver shows.
- A wrong password simply gives `401 Unauthorized`. There is no retry loop as in Java.

C15. Reading a password (Java B15)

Go's standard library has no GUI, so there is no JPasswordField equivalent. Command-line tools use the official `golang.org/x/term` package, which hides what is typed and also returns bytes that can be wiped:

```
password, err := term.ReadPassword(int(os.Stdin.Fd()))  // []byte, nothing echoed
```

It is an extra dependency, so it is shown here but not included as a program in this unit.

C16. Interview notes (Go)

Q. What is wrong with http.Get in production code?

http.DefaultClient has **no timeout**, so a stalled server can hang a goroutine forever. Create `&http.Client{Timeout: 10*time.Second}`, or use a request with a context.

Q. Why must you close resp.Body?

The connection returns to the pool only after the body is closed, and fully read for reuse. Forgetting leaks connections and file descriptors. `defer` it after checking err.

Q. Does http.Get return an error for a 404?

No. err is only for transport failures such as DNS, connection or timeout. HTTP error statuses arrive as a normal response, so check resp.StatusCode.

Q. How do you safely build a URL with user input?

Never concatenate strings. Use url.Values with Encode() for the query, and url.PathEscape or the url.URL struct's Path field for the path. Otherwise a value containing & or # changes the URL's meaning.

Q. What is SSRF, and how do you defend against it?

Server-Side Request Forgery: an attacker supplies a URL and your server fetches it, reaching internal services such as cloud metadata at 169.254.169.254. Allowlist schemes and hosts, then check the resolved IP with IsPrivate and IsLoopback, using a custom Dialer so the check can't be bypassed by DNS changes.

Q. Why is parsing a URL with url.Parse not enough to validate it?

Parse only checks syntax and is lenient: it accepts unknown schemes, and even a bare word parses as a path. Check u.Scheme and u.Host explicitly.

Q. How do proxies work for HTTPS?

The client sends `CONNECT host:443`. The proxy opens a raw TCP tunnel and the TLS handshake happens through it, so the proxy cannot read the traffic. For plain HTTP, the proxy sees the full request.

Q. Basic auth, bearer tokens or cookies: when would you use each?

Basic for simple internal tools over HTTPS. Bearer tokens (JWT or opaque) for APIs: Authorization: Bearer ..., and they can expire and be revoked. Cookies for browser sessions, with HttpOnly, Secure and SameSite set.

Part D: Likely exam questions for Unit 3

These are typical of how Unit 3 is asked. Check them against past CACS355 papers and mark the ones that repeat.

Short questions

#	Question	Answer pointer
1	Differentiate URI, URL and URN with examples.	A1 table
2	Explain the parts of a URL with an example.	A1 diagram and table
3	What is a relative URL? Why is it useful?	A2
4	List the constructors of the URL class.	B2 table
5	Explain the methods for retrieving data from a URL.	B3 table
6	Differentiate getFile() and getPath(); equals() and sameFile().	B4, B5 tables
7	Why should URL objects not be used in a HashMap?	B5 (DNS in equals/hashCode)
8	Differentiate the URL and URI classes.	B6 table
9	Explain resolve(), relativize() and normalize().	B8 table
10	What is x-www-form-urlencoded? Explain URLEncoder and URLDecoder.	A3, B10
11	What is a proxy server? Why is it used?	A4
12	List the system properties used to set a proxy.	B11 table
13	Explain the Authenticator and PasswordAuthentication classes.	B14
14	Why does JPasswordField return a char array?	B15

Programs (long questions)

#	Write a Java program to...	Program
1	split a URL into protocol, host, port, path, query and fragment	B4 URLSplitter
2	test which protocols the virtual machine supports	B2 ProtocolTester
3	download and display the source of a web page	B3 SourceViewer
4	show the parts of a URI	B7 URISplitter
5	resolve a relative URI against a base URI	B8 URIResolver
6	encode and decode a string using URLEncoder and URLDecoder	B10 EncoderDecoder
7	build a query string and send it to a server using GET	B10 QueryString, B13 FormGet
8	connect through a proxy using the Proxy class	B11 ProxyDemo
9	implement a custom ProxySelector	B12 LocalProxySelector
10	access a password-protected site using Authenticator	B14 SecureViewer
11	ask for a password with a dialog using JPasswordField	B15 DialogAuthenticator

Answer-writing pattern for program questions

1. One line stating the idea. 2. The program, with imports and the right exception. 3. Comments on the key calls. 4. A sample output box.

Part E: Java vs Go cheat sheet (Unit 3)

Task	Java	Go
Parse	`new URL(s)`, `new URI(s)`	`url.Parse(s)`
Resolve relative	`base.resolve(s)`	`base.Parse(s)`
Fetch	`u.openStream()`	`http.Get(s)`
Scheme	`getProtocol()`, `getScheme()`	`u.Scheme`
Host	`getHost()`	`u.Hostname()`
Port	`getPort()` (-1)	`u.Port()` ("")
Path	`getPath()`	`u.Path`
Path + query	`getFile()`	`u.RequestURI()`
Query	`getQuery()`	`u.RawQuery`, `u.Query()`
Fragment	`getRef()`, `getFragment()`	`u.Fragment`
Opaque?	`isOpaque()`	`u.Opaque != ""`
Absolute?	`isAbsolute()`	`u.IsAbs()`
Same file	`sameFile(u)`	write a helper
To text	`toString()`	`u.String()`
Encode	`URLEncoder.encode(s, "UTF-8")`	`url.QueryEscape(s)`
Decode	`URLDecoder.decode(s, "UTF-8")`	`url.QueryUnescape(s)`
Build query	QueryString class	`url.Values{}`, `Encode()`
Proxy for all	System properties	HTTP_PROXY variable
Proxy for one	`openConnection(proxy)`	`Transport{Proxy: http.ProxyURL(u)}`
Proxy chooser	`extends ProxySelector`	`func(*http.Request) (*url.URL, error)`
Basic login	`Authenticator.setDefault(...)`	`req.SetBasicAuth(u, p)`
Bad URL	`MalformedURLException`	`err != nil`

Next: Unit 4, HTTP (the protocol, Keep-Alive, HTTP methods, the request body, CookieManager and CookieStore), in the same Java-then-Go layout.